

redrob IDEA SUBMISSION TEMPLATE

MeritEngine.ai — Empathy-First Candidate Discovery & Ranking (Track 01)

Team Name:	team_meritengine
Team Leader:	Suraj Das
Primary Contact:	etcporasonaetc@gmil.com +91 83360 41207
Code Repository:	https://github.com/qitsuraj7/meritengine

01. Solution Overview

What is your proposed solution?

- A hybrid **two-stage candidate evaluation engine** that balances sub-second parsing speed and deep semantic ranking.
- **Stage 1:** Filters 100K profiles line-by-line in under 10 seconds to isolate the top 1,000 candidates based on job spec compatibility and availability.
- **Stage 2:** Deeply scores the top candidates using a 10-layer, 50-agent Empathy Judging Panel, saving high-potential self-made developers from immediate rejection.

What differentiates your approach from traditional systems?

- Traditional applicant tracking systems rely on static keyword parsing and FAANG/Ivy credentials.
- MeritEngine discounts empty brand credentials, tracks contribution velocities/streaks, and utilizes multi-agent advocacy overlays to highlight non-traditional builders.

02. JD Understanding & Candidate Evaluation

Key Requirements Extracted from the JD:

- **Experience:** 5-9 years of applied machine learning / information retrieval experience in product settings.
- **Tech Stack:** Strong Python. Hands-on vector databases (Milvus, Pinecone, OpenSearch) and evaluation metrics (NDCG, MAP, MRR).
- **Disqualifiers:** Pure research/academic backgrounds, LangChain-only hobbyists, and consulting-only careers (TCS, Wipro, Infosys).

How our solution evaluates fit beyond keyword matching:

- **Semantic Fit Matcher:** Embeds candidate summaries and descriptions to match the semantic intent of the job description.
- **Active Availability Adjustments:** Prioritizes high recruiter response rates and penalizes long notice periods (> 90 days).

03. Ranking Methodology

How does the system retrieve, score, and rank candidates?

- **Stage 1 (Vectorized Heuristic Filter):** Processes all 100,000 JSON records, penalizes consulting backgrounds and non-tech titles, and keeps the top 1,000.
- **Stage 2 (Pydantic Evaluation & Empathy Boost):** Deeply evaluates candidate dimensions and applies a soft boost.

What models, algorithms, or heuristics are used?

- **Honeypot Filter:** Blocks profiles exhibiting expert skills with zero months usage.
- **Empathy Committee:** A 10-layer, 50-agent soft-score overlay panel providing up to +15 overall score points based on grit markers.
- **Tie-Breaking Rule:** Deterministically resolves score ties using `candidate\_id` ascending as specified in Section 3.

How are multiple candidate signals combined into a final ranking?

- **Composite Matrix Matching:** Aggregates skill claimed matching, notice period and salary alignment discounts, and public commit velocities into a unified, normalized score.

04. Explainability

How are ranking decisions explained?

- **Qualitative Advocacy Note:** The Ombudsman layer compiles specific reasons (e.g. 'Data Scientist with 6.8 YoE, strong OpenSearch, 90% response rate').
- **Reasoning monograms:** Stored in the CSV file for manual recruiter review.

How do you prevent hallucinations or unsupported justifications?

- String templates are built dynamically using verified, non-hallucinated properties inside the candidate's verified profile data (no external generative model hallucination).

Handling inconsistent, low-quality, or suspicious profiles:

- Accounts flag and penalize candidates with mismatching work history dates, empty profiles, or inconsistent skill proficiency metrics.

05. End-to-End Workflow

Complete flow from raw data input to ranked candidate output:

1. Raw Ingestion	Parse raw candidates.jsonl file line-by-line (sub-second per profile).
2. Heuristic Filtering	Prune honeypots, consulting profiles, and non-tech titles. Compute heuristic scores.
3. Deep Evaluation	Convert top 1,000 to Pydantic objects. Run dimension scoring and Empathy Committee boost.
4. Sorting & Formatting	Sort by overall score descending, break ties deterministically, and export the top 100 candidates.
5. Compliance Check	Run official validate_submission.py to ensure zero spec violations.

06. System Architecture

Pipeline Module Structure:

Module	Function & Architecture role
meritengine/core/models.py	Pydantic v2 domain schemas modeling Candidate, RoleSpec, and Verdict.
meritengine/core/scoring/committee.py	Main Empathy Committee containing the 10 chambers and 50 sub-agents.
meritengine/core/pipeline.py	Orchestrates dimension scorers and overlays committee qualitative boosts.
rank.py	The optimized two-stage batch ranking execution script complying with compute constraints.

07. Results & Performance

What results or insights demonstrate ranking quality?

- **Relevance Output:** Top 10 ranks feature candidates matching the experience band (5-9 years) who have built search/retrieval platforms at product-based companies.
- **Honeypot Rate:** **0% honeypots** in our top 100 shortlist, satisfying Stage 3 compliance guidelines.

How our solution meets runtime and compute constraints:

- **Runtime Limit:** processes 100K candidates and exports the shortlist in **under 15 seconds** (5-minute budget limit).
- **Resource Limits:** CPU-only, 100% offline (no OpenAI/Gemini API calls), and uses less than 1 GB RAM (16 GB limit).

08. Technologies Used

Why were these technologies selected?

Technology	Selection Rationale
Python 3.12	Core language, robust parsing, and native zip handling.
Pydantic v2	Strict validation, lightning-fast parsing, and model dump capabilities.
SQLite3	Zero-configuration local DB providing persistent pipeline state tracking.
ReportLab	Professional PDF rendering to compile simulation logs and presentation slide decks.

09. Submission Assets

Access links and files compiled for Stage 3 validation:

Submission CSV Shortlist:	team_meritengine.csv (exactly 100 rows, valid)
Methodology Metadata:	submission_metadata.yaml (filled and tested)
Source Code Repository:	https://github.com/gitsuraj7/meritengine
Execution Command:	<code>python rank.py --candidates ./candidates.jsonl --out ./team_meritengine.csv</code>
PDF Presentation Slides:	presentation_slides.pdf (this file)

THANK YOU

redrob

Presented by Team MeritEngine | Building with Empathy